

Custom Time Series Event Detection in R with Harbinger



Roadmap

Harbinger is more than a catalog of built-in methods. It is a framework with clear contracts for transformations, detectors, motif flows, and evaluators. This slide series introduces each extension point progressively.

01

Transformation

Custom median filter

03

Sequence Anomaly

Collective detector with intervals

05

Motif

DTW subsequence clustering

02

Anomaly Detector

Wrapper for `forecast::tsoutliers()`

04

Change Point

Joinpoint regression with `segmented`

06

Evaluation

Custom range-aware evaluator

Custom Transformation: Median Filter

Raw time series often contain isolated spikes caused by sensor noise, transmission errors, or measurement artifacts. If fed directly into a detector, these spikes can trigger false positives or mask true anomalies. Applying a preprocessing transformation cleans the signal before detection, improving both precision and interpretability.

Goal: apply `stats::runmed()` to the series before the detector, smoothing isolated spikes without distorting trends.

```
library(daltoolbox)
library(harbinger)

har_fil_median_custom <- function(k = 5) {
  if (k %% 2 == 0) k <- k + 1
  obj <- daltoolbox::dal_transform()
  obj$k <- k
  class(obj) <- append(
    "har_fil_median_custom", class(obj))
  obj
}
```

```
transform.har_fil_median_custom <-
function(obj, data, ...) {
  result <- stats::runmed(
    as.numeric(data),
    k = obj$k,
    endrule = "keep")
  result[is.na(result)] <-
    as.numeric(data)[is.na(result)]
  result
}

# Uso
filter_custom <- har_fil_median_custom(k = 5)
serie_filtered <- transform(
  filter_custom, dataset$serie)
```

Step by Step

- Ensure `k` is odd (required by `runmed`)
- Create base object with `dal_transform()`
- Append custom class with `append()`
- Implement `transform()`: apply `runmed()` and preserve NAs
- Use the object exactly like any native transformation

📌 **Key concept:** A custom transformation only needs a constructor and a `transform()` method. The rest of the workflow remains unchanged.

Applying the Transformation in the Pipeline

After filtering the series, the ARIMA detector is fed the smoothed signal. The median filter is a pre-processing step, not a detector.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple

# Visualizar série bruta
har_plot(harbinger(),
  dataset$serie,
  event = dataset$event)

# Aplicar filtro e visualizar
filter_custom <- har_fil_median_custom(k =
5)
serie_filtered <- transform(
  filter_custom, dataset$serie)

har_plot(harbinger(),
  serie_filtered,
  event = dataset$event)
```

```
# Detectar com ARIMA sobre série filtrada
model <- hanr_arima()
model <- fit(model,
  as.numeric(serie_filtered))
detection <- detect(model,
  as.numeric(serie_filtered))

har_plot(model,
  as.numeric(serie_filtered),
  detection,
  dataset$event)
```

Step by Step

- Load data with `examples_anomalies$simple`
- Plot the raw series as a visual reference
- Apply `transform()` to obtain the filtered series
- Plot the filtered series and compare with the raw one
- Fit `hanr_arima()` on the filtered series
- Detect and plot the final result

📌 **Key concept:** Transformation → Detection. Changing the signal representation first improves the interpretability of the downstream detector.

Pointwise Anomaly Detection: Concepts & Integration

A pointwise anomaly is an individual observation that deviates significantly from the expected behavior of the series. Detecting them requires a model of normality — a baseline against which each point is compared. Harbinger's open architecture allows any external detection library to be wrapped as a native detector, as long as it respects a minimal two-part contract.

Pointwise Anomaly

A single timestamp where the observed value is statistically inconsistent with the surrounding context. Common causes: sensor faults, data entry errors, sudden real-world events.

`tsoutliers()` (forecast pkg)

Detects additive outliers (AO) and temporary changes (TC) by fitting an ARIMA model and identifying residuals that exceed a threshold. Returns the index and replacement value for each outlier.

Detector Contract

In Harbinger, a custom detector needs only: (1) a constructor based on `harbinger()`, and (2) a `detect()` method that returns a standardized detection table via `har_restore_refs()`. No reimplementation of plotting or evaluation is needed.

Further reading: Ogasawara, E. et al. (2025). Event Detection in Time Series. Springer. — Chapter on pointwise anomaly detectors and the Harbinger detector contract.

Custom Detector: forecast::tsoutliers()

Goal: wrap `forecast::tsoutliers()` as an anomaly detector in Harbinger, identifying additive outliers and temporary disturbances.

```
library(daltoolbox)
library(harbinger)
# install.packages("forecast")

hanr_tsoutliers_custom <-
  function(frequency = 1) {
    obj <- harbinger()
    obj$frequency <- frequency
    class(obj) <- append(
      "hanr_tsoutliers_custom", class(obj))
    obj
  }
```

```
detect.hanr_tsoutliers_custom <-
  function(obj, serie, ...) {
    obj <- obj$har_store_refs(obj, serie)

    y_ts <- stats::ts(obj$serie,
                      frequency = obj$frequency)
    out <- forecast::tsoutliers(y_ts)

    anomalies <- rep(FALSE, length(obj$serie))
    if (!is.null(out$index) &&
        length(out$index) > 0) {
      anomalies[unique(out$index)] <- TRUE
    }

    obj$har_restore_refs(obj,
                        anomalies = anomalies)
  }
```

Step by Step

- Create constructor based on `harbinger()`
- Store frequency in the object
- Implement `detect()`: convert series to `ts`
- Call `forecast::tsoutliers()` and capture indices
- Map positions to a logical anomaly vector
- Return with `har_restore_refs()`

📌 **Key concept:** The integration contract is minimal — constructor + `detect()` method. The external library decides the suspicious timestamps; Harbinger standardizes the result.

Running and Evaluating the Custom Detector

Once the detector is defined, its usage is identical to any native Harbinger detector: `detect()`, `evaluate()`, and `har_plot()`.

```
data(examples_anomalies)
dataset <- examples_anomalies$simple

model <- hanr_tsoutliers_custom()
detection <- detect(model, dataset$serie)

# Avaliar
evaluation <- evaluate(model,
                        detection$event,
                        dataset$event)
evaluation$confMatrix
```

```
## event
## detection TRUE FALSE
##   TRUE  0   0
##   FALSE 1 100

# Visualizar
har_plot(model,
          dataset$serie,
          detection,
          dataset$event)
```

Step by Step

- Instantiate with `hanr_tsoutliers_custom()`
- Call `detect(model, serie)` — no `fit()` required
- Evaluate with `evaluate()` and inspect `confMatrix`
- Plot with `har_plot()` for visual inspection

📌 **Key concept:** A custom detector does not need to re-implement the full workflow. Respecting the constructor + `detect()` contract is enough to reuse native plots and evaluation.

Confusion matrix cells: **TP** — real anomaly correctly detected; **FP** — normal point incorrectly flagged; **FN** — real anomaly missed; **TN** — normal point correctly left unflagged.



Extending Harbinger: Custom Change Point & Motif Detectors

What is a Change Point?

A change point signifies a moment in a time series where the underlying statistical properties, such as slope, mean, or variance, undergo an abrupt shift. Identifying these points is crucial for understanding system dynamics and predicting future behavior.



Identifying Shifts

Detects abrupt changes in statistical properties (e.g., mean, variance, slope) within time series data.



Segmented Regression

Utilizes joinpoint regression to fit piecewise linear models, precisely locating trend breakpoints.



Harbinger's Approach

Implements a 3-method contract: `constructor`, `fit()`, and `detect()` for seamless integration.

Common use cases include detecting trend reversals in sensor data, economic indicators, or clinical measurements, enabling timely interventions or insights.

 Reference: Muggeo, V.M.R. (2003). Estimating regression models with unknown break-points. *Statistics in Medicine*, 22(19), 3055–3071.

Custom Detector: Joinpoint Regression

Goal: detect points where a linear trend changes direction or slope, using `segmented::segmented()` as the estimation engine.

```
library(daltoolbox)
library(harbinger)
# install.packages("segmented")

hcp_joinpoint_custom <- function(npsi = 1) {
  obj <- harbinger()
  obj$npsi <- npsi
  class(obj) <- append(
    "hcp_joinpoint_custom", class(obj))
  obj
}

fit.hcp_joinpoint_custom <-
function(obj, data, ...) {
  x <- seq_along(data)
  base_model <- stats::lm(data ~ x)
  obj$model <- segmented::segmented(
    base_model,
    seg.Z = ~ x,
    npsi = obj$npsi)
  obj
}
```

```
detect.hcp_joinpoint_custom <-
function(obj, serie, ...) {
  obj <- obj$har_store_refs(obj, serie)
  if (is.null(obj$model))
    obj <- fit(obj, obj$serie)

  cp <- rep(FALSE, length(obj$serie))
  psi <- round(obj$model$psi[, "Est."])
  psi <- psi[psi >= 1 &
    psi <= length(cp)]
  cp[psi] <- TRUE

  obj$har_restore_refs(obj,
    change_points = cp)
}
```

Step by Step

- Constructor stores `npsi` (number of joinpoints)
- `fit()`: fit a base linear model with `lm(data ~ x)`
- Refine with `segmented()` to estimate breakpoints
- `detect()`: extract estimated positions from `$psi[, "Est."]`
- Round and validate indices within series bounds
- Mark positions as `change_points = TRUE`

📌 **Key concept:** Joinpoint regression connects change detection with regression interpretation — the breakpoint is where one straight line ends and another begins.

Fitting, Evaluating, and Visualizing the Joinpoint

The joinpoint detector reuses Harbinger's standard evaluation and plot workflow after converting the estimated breakpoint into the common detection table.

```
data(examples_changepoints)
dataset <- examples_changepoints$simple

model <- hcp_joinpoint_custom(npsi = 1)
model <- fit(model, dataset$serie)

## Warning: essentially perfect fit:
## summary may be unreliable

detection <- detect(model, dataset$serie)

# Avaliar
evaluation <- evaluate(model,
                       detection$event,
                       dataset$event)
evaluation$confMatrix
```

```
## event
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

# Visualizar
har_plot(model,
         dataset$serie,
         detection,
         dataset$event)
```

Step by Step

- Load `examples_changepoints$simple`
- Instantiate with `npsi = 1` (one joinpoint)
- Call `fit()` explicitly before `detect()`
- The "perfect fit" warning is expected for simple synthetic series
- Confusion matrix: 1 TP, 0 FP, 0 FN — perfect detection
- Plot to visually confirm the breakpoint

📌 **Key concept:** Fit structural model → convert estimate to detection table → reuse native evaluation and plot. This is the standard change-point extension pattern.

What is a Motif?

Motif detection focuses on identifying recurring subsequence patterns within time series data. Unlike anomaly detection, which seeks out the unusual, motif detection uncovers the most common and representative "signatures" or "shapes" that appear repeatedly.



What is a Motif?

Recurring subsequence patterns, identifying typical behaviors rather than anomalies.



Why DTW?

Dynamic Time Warping (DTW) enables elastic alignment, recognizing similar patterns despite speed or phase shifts.



How Harbinger Wraps It

Leverages a 3-method contract (`constructor`, `fit()`, `detect()`), where `fit()` clusters and `detect()` selects representative instances.

Common applications include identifying recurring behaviors in ECG signals, detecting typical cycles in industrial machinery, or recognizing seasonal patterns in sensor data.

Reference: Sardá-Espinosa, A. (2019). Time-series clustering in R using the dtwclust package. The R Journal, 11(1), 22–43.

Motif Detector: DTW Clustering – Constructor and Fit

Goal: discover motifs as families of similar subsequences via DTW clustering, instead of nearest-neighbor search through Matrix Profile.

```
library(daltoolbox)
library(harbinger)
# install.packages("dtwclust")

hmo_dtw_cluster_custom <-
  function(w = 20, centers = 3,
           min_cluster_size = 3) {
    obj <- harbinger()
    obj$w <- w
    obj$centers <- centers
    obj$min_cluster_size <- min_cluster_size
    class(obj) <- append(
      "hmo_dtw_cluster_custom", class(obj))
    obj
  }
```

```
fit.hmo_dtw_cluster_custom <-
  function(obj, data, ...) {
    windows <- tspredit::ts_data(data, obj$w)
    subsequences <- lapply(
      seq_len(nrow(windows)),
      function(i) as.numeric(windows[i, ]))

    obj$model <- dtwclust::tsclust(
      series = subsequences,
      type = "partitional",
      k = obj$centers,
      distance = "dtw_basic",
      centroid = "dba",
      seed = 1L,
      trace = FALSE)
    obj
  }
```

Step by Step

- Parameters: `w` (window size), `centers` (clusters), `min_cluster_size`
- `fit()`: extract sliding windows with `ts_data()`
- Convert each row into a list of numeric vectors
- Cluster with `tsclust()`: DTW distance, DBA centroid
- Store clustering model in `obj$model`

❏ **Key concept:** DTW allows elastic alignment — two similar patterns at different speeds are recognized as the same motif.

Motif Detector: `detect()` Method and Execution

The `detect()` method selects populous clusters and keeps non-overlapping representatives to prevent a single motif family from dominating the plot.

```
detect.hmo_dtw_cluster_custom <-  
function(obj, serie, ...) {  
  n <- length(serie)  
  non_na <- which(!is.na(serie))  
  serie_c <- stats::na.omit(serie)  
  if (is.null(obj$model))  
    obj <- fit(obj, serie_c)  
  
  cluster_id <- obj$model@cluster  
  cluster_sizes <- table(cluster_id)  
  selected <- names(  
    cluster_sizes[cluster_sizes >=  
      obj$min_cluster_size])  
  
  out <- data.frame(  
    event = rep(FALSE, length(serie_c)),  
    seq = rep(NA, length(serie_c)))
```

```
seq_counter <- 1  
for (cl in selected) {  
  positions <- which(  
    cluster_id == as.integer(cl))  
  sel <- integer(0); last <- -Inf  
  for (pos in positions) {  
    if ((pos - last) >= obj$w) {  
      sel <- c(sel, pos); last <- pos  
    }  
  }  
  out$event[sel] <- TRUE  
  out$seq[sel] <- as.character(seq_counter)  
  seq_counter <- seq_counter + 1  
}  
# montar detection e retornar ...  
}  
  
# Uso  
model <- hmo_dtw_cluster_custom(  
  w=15, centers=3, min_cluster_size=3)  
model <- fit(model, dataset$serie)  
detection <- detect(model, dataset$serie)  
har_plot(model, dataset$serie,  
  detection, dataset$event)
```

Step by Step

- Get cluster assignment: `obj$model@cluster`
- Filter clusters with size $\geq$ `min_cluster_size`
- For each selected cluster, choose non-overlapping representatives (distance $\geq w$)
- Mark positions as `event = TRUE` with sequence ID
- Build detection `data.frame` with `idx`, `event`, `type`, `seq`, `seqlen`

 **Key concept:** Recurring clusters = motif families. Non-overlapping representatives ensure the plot remains readable and informative.

Custom Evaluation: Pointwise vs. Range-Aware

Standard pointwise metrics treat each timestamp independently — a detection is only correct if it precisely matches the ground truth at that exact point. For collective or sequence anomalies that span multiple timestamps, this is inherently unfair: a slightly misaligned but heavily overlapping detection is counted as entirely missed, penalizing models that are actually performing well.

The range-aware evaluator gives partial credit to detections that overlap the true interval, unlike pointwise evaluation which can be unfair for collective anomalies.

```
# Exemplo didático com intervalos conhecidos
n <- 100
event <- rep(FALSE, n)
detection <- rep(FALSE, n)

# Dois intervalos verdadeiros
event[20:30] <- TRUE
event[60:75] <- TRUE

# Uma detecção parcial, uma boa, uma falsa
detection[18:27] <- TRUE
detection[64:78] <- TRUE
detection[88:92] <- TRUE

# Avaliação pontual (dura)
hard_eval <- evaluate(har_eval(), detection, event)
hard_eval$confMatrix

## event
## detection TRUE FALSE
##      TRUE  20  10
##      FALSE   7  63
```

```
# Avaliação range-aware (crédito parcial)
range_eval <- evaluate(
  har_eval_range_custom(alpha = 0.5),
  detection, event)

## $precision [1] 0.5333333
## $recall   [1] 0.8693182
## $F1       [1] 0.6610856
## $alpha    [1] 0.5
```

Step by Step

- Create event and detection vectors with known intervals
- Evaluate with `har_eval()`: strict pointwise metric
- Evaluate with `har_eval_range_custom(alpha=0.5)`: partial credit
- Compare: pointwise ignores partial overlap; range-aware rewards it
- Precision 0.53, Recall 0.87, F1 0.66 — better reflects actual quality

❏ **Key concept:** The choice of metric must reflect the anomaly type. Sequence anomalies require range-aware metrics — pointwise metrics can be misleading.



Collective Anomaly Detection: Sequence Ranges and Evaluation

Sequence Detector: Custom Rolling MAD

A **point anomaly** is a single observation that significantly deviates from the norm. A **collective anomaly**, by contrast, is a contiguous subsequence where individual points may not be extreme, but the overall pattern is anomalous. Returning the full anomalous **interval** — rather than a single representative timestamp — is essential for understanding the duration and extent of such events.

Goal: detect collective anomalies by returning the full interval, not just a representative timestamp. Uses a rolling median baseline and local MAD scaling.

```
hanr_sequence_mad_custom <-  
  function(window_size = 11,  
           z_threshold = 3,  
           min_run = 3) {  
    obj <- harbinger()  
    obj$window_size <- window_size  
    obj$z_threshold <- z_threshold  
    obj$min_run <- min_run  
    class(obj) <- append(  
      "hanr_sequence_mad_custom", class(obj))  
    obj  
  }  
  
detect.hanr_sequence_mad_custom <-  
  function(obj, serie, ...) {  
    obj <- obj$har_store_refs(obj, serie)  
    y <- as.numeric(obj$serie)  
    k <- obj$window_size  
    if (k %% 2 == 0) k <- k + 1  
  
    baseline <- zoo::rollmedian(  
      y, k = k, fill = NA, align = "center")  
    local_mad <- zoo::rollapply(  
      y, width = k,  
      FUN = function(v) stats::mad(v, constant=1),  
      fill = NA, align = "center")
```

```
    baseline <- zoo::na.locf(baseline,  
                           na.rm=FALSE, fromLast=FALSE)  
    baseline <- zoo::na.locf(baseline,  
                           fromLast=TRUE, na.rm=FALSE)  
    local_mad <- zoo::na.locf(local_mad,  
                           na.rm=FALSE, fromLast=FALSE)  
    local_mad <- zoo::na.locf(local_mad,  
                           fromLast=TRUE, na.rm=FALSE)  
    local_mad[local_mad == 0 |  
              is.na(local_mad)] <- 1e-8  
  
    score <- abs(y - baseline) / local_mad  
    raw_flags <- score > obj$z_threshold  
    # ... run grouping (see next slide)  
  }
```

Step by Step

- Ensure `k` is odd for `rollmedian`
- Compute baseline with rolling median (`zoo::rollmedian`)
- Compute local scale with rolling MAD (`zoo::rollapply`)
- Fill edges with `na.locf` (forward and backward)
- Avoid division by zero: replace `MAD=0` with `1e-8`
- Compute robust z-score: $|y - \text{baseline}| / \text{mad}$

📌 **Key concept:** The local robust z-score is more resistant to outliers than the classical z-score, as it uses median and MAD instead of mean and standard deviation.

Run Grouping and Execution

The core customization: instead of keeping only one point per group, the detector marks the entire contiguous interval as anomalous.

```
# Agrupar runs contíguos de TRUE
idx_true <- which(raw_flags)
runs <- if (length(idx_true) == 0) {
  list(integer(0))
} else {
  split(idx_true,
        cumsum(c(1, diff(idx_true) != 1)))
}
range_flags <- rep(FALSE, length(y))
for (run in runs) {
  if (length(run) >= obj$min_run) {
    range_flags[run] <- TRUE # intervalo completo
  }
}
detection <- obj$har_restore_refs(
  obj, anomalies = range_flags, res = score)
attr(detection, "threshold") <- obj$z_threshold
detection
}
```

```
# Uso
model <- hanr_sequence_mad_custom(
  window_size = 9,
  z_threshold = 2.5,
  min_run = 2)
detection <- detect(model, dataset$serie)
head(detection[detection$event, ])

## idx event  type
## 51  51 TRUE anomaly
## 52  52 TRUE anomaly
```

Step by Step

- Identify TRUE indices with `which(raw_flags)`
- Group contiguous runs with `split + cumsum + diff`
- Filter runs with `length ≥ min_run`
- Mark **all** points in the run as anomalous
- Return detection with `threshold` attribute

📌 **Key concept:** Sequence anomalies are best represented as intervals. Collapsing a segment to a single point hides the duration of the abnormal behavior.

Custom Evaluator: Range-Aware Metrics

Evaluating collective anomaly detectors requires a nuanced approach that moves beyond traditional point-based metrics. Our framework focuses on range overlap to provide a more accurate assessment of performance.

Problem with Point-Based Metrics

Classical metrics (precision, recall, F1) are inadequate for interval-based anomalies. A detector that flags a single point within a true anomaly interval receives zero precision and recall, even if it accurately identified the anomalous region. This penalizes relevant detections too harshly.

Range-Based Approach

Both detected outputs and ground truth are converted into contiguous intervals (ranges). Evaluation then focuses on the degree of overlap between these detected ranges and the true anomaly ranges.

Range-Based Precision

For each detected range, range-based precision quantifies the fraction that overlaps with any true anomaly range. This metric penalizes detections that are too broad or entirely incorrect.

Range-Based Recall & Alpha

For each true anomaly range, recall determines if it was found. This combines an **existence reward** (was any detection nearby?) and an **overlap reward** (how much of the true range was covered?). The **alpha parameter** ($\alpha \in [0,1]$) controls this balance:

- High alpha rewards existence (finding the anomaly at all).
- Low alpha rewards precise overlap.
- Alpha = 0.5 balances both aspects.

Range-Based F1 Score

The F1 score is calculated as the harmonic mean of range-based precision and recall, providing a single metric that reflects the overall effectiveness of the detector.

Range-Aware Evaluator: Support Structures

To evaluate intervals, a custom evaluator is defined that converts boolean vectors into ranges and computes precision/recall based on overlap.

```
har_eval_range_custom <- function(alpha = 0.5) {  
  obj <- daltoolbox::dal_base()  
  obj$alpha <- alpha  
  class(obj) <- append(  
    "har_eval_range_custom", class(obj))  
  obj  
}  
  
range_overlap <-  
  function(a_start, a_end, b_start, b_end) {  
    max(0, min(a_end, b_end) -  
      max(a_start, b_start) + 1)  
  }
```

```
to_ranges <- function(flags) {  
  flags <- as.logical(flags)  
  flags[is.na(flags)] <- FALSE  
  idx <- which(flags)  
  if (length(idx) == 0)  
    return(data.frame(start=integer(0),  
                      end=integer(0),  
                      length=integer(0)))  
  groups <- split(idx,  
    cumsum(c(1, diff(idx) != 1)))  
  starts <- vapply(groups, min, integer(1))  
  ends <- vapply(groups, max, integer(1))  
  data.frame(start=starts, end=ends,  
    length=ends - starts + 1)  
}
```

Step by Step

- Constructor based on `dal_base()` with `alpha` parameter
- `to_ranges()`: converts a logical vector into a `data.frame` of intervals (start, end, length)
- Group contiguous TRUE indices with `split + cumsum`
- `range_overlap()`: computes overlap between two intervals
- Returns 0 if there is no overlap

📌 **Key concept:** `to_ranges()` and `range_overlap()` are reusable helper functions for any range-aware evaluator in Harbinger.

Range-Aware Evaluator: Precision and Recall

The `evaluate()` method combines existence and overlap rewards, controlled by the `alpha` parameter.

```
evaluate.har_eval_range_custom <-  
function(obj, detection, event, ...) {  
  det_ranges <- to_ranges(detection)  
  evt_ranges <- to_ranges(event)  
  
  if (nrow(det_ranges)==0 ||  
      nrow(evt_ranges)==0)  
    return(list(precision=0, recall=0, F1=0))  
  
  recall_scores <- numeric(nrow(evt_ranges))  
  for (i in seq_len(nrow(evt_ranges))) {  
    overlaps <- mapapply(range_overlap,  
      evt_ranges$start[i], evt_ranges$end[i],  
      det_ranges$start, det_ranges$end)  
    best <- max(overlaps)  
    exist <- as.numeric(best > 0)  
    overl <- best / evt_ranges$length[i]  
    recall_scores[i] <-  
      obj$alpha * exist +  
      (1 - obj$alpha) * overl  
  }  
}
```

```
prec_scores <- numeric(nrow(det_ranges))  
for (i in seq_len(nrow(det_ranges))) {  
  overlaps <- mapapply(range_overlap,  
    det_ranges$start[i], det_ranges$end[i],  
    evt_ranges$start, evt_ranges$end)  
  prec_scores[i] <-  
    max(overlaps) / det_ranges$length[i]  
}  
  
precision <- mean(prec_scores)  
recall <- mean(recall_scores)  
F1 <- if ((precision+recall)==0) 0 else  
  2*precision*recall/(precision+recall)  
list(precision=precision,  
     recall=recall, F1=F1)  
}
```

Step by Step

- Convert detection and ground truth to ranges
- **Recall:** for each true range, compute the best overlap with detected ranges
- Combine existence reward (`alpha`) and overlap reward (`1-alpha`)
- **Precision:** for each detected range, fraction that overlaps any true range
- Compute harmonic F1 between precision and recall

❏ **Key concept:** `alpha` controls the trade-off — high values reward finding the range; low values reward precise overlap.



Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

